From the Source to a Flat Table

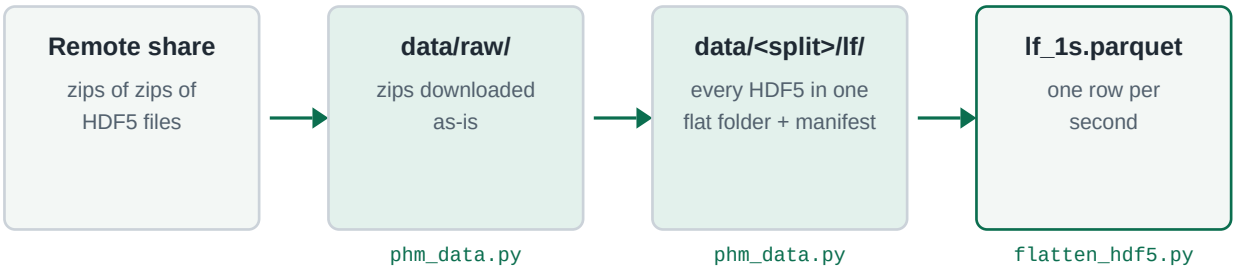
How the PHM 2026 gear-test data is organized where it lives, and how two scripts turn it into one-row-per-second parquet files

The short version

The data comes from a gearbox test rig. A gear is run on a dynamometer (a machine that spins it under load) until it wears out. While it runs, sensors record vibration, speed, torque, temperature, and the amount of metal debris in the oil. Every minute or so the rig writes a small file. Over one test there are thousands of these files.

At the source, the files are packed in zip archives that sit inside other zip archives, and each file is itself a container format (HDF5) with several groups of numbers inside. That is a lot of nesting for analysis. Two scripts undo the nesting:

- **scripts/phm_data.py** downloads the archives, unzips them twice, and lays every file out in one flat folder per data type, with a manifest that records where each file came from.
- **scripts/flatten_hdf5.py** opens every HDF5 file, lines up all the sensor readings on a shared one-second clock, and writes two parquet tables per split: **lf_1s.parquet** (one row per second) and **particles.parquet** (one row per file).



The rest of this document walks through each stage in order: what the source looks like, what one raw file holds, what each script does, and what the final tables contain.

1. Where the data lives

The challenge organizers host the data on a Synology file server. There are three separate shares, one for each split. Each share has the same folder layout.

Split	Experiments	Used for	Login
train	Exp-A, Exp-B, Exp-C, Exp-F	Building models. Full run-to-failure histories with photos.	open
validation	Exp-G	Checking models. Comes with an example submission file.	open
test	Exp-D, Exp-E	Final scoring. Comes with an example submission file.	password

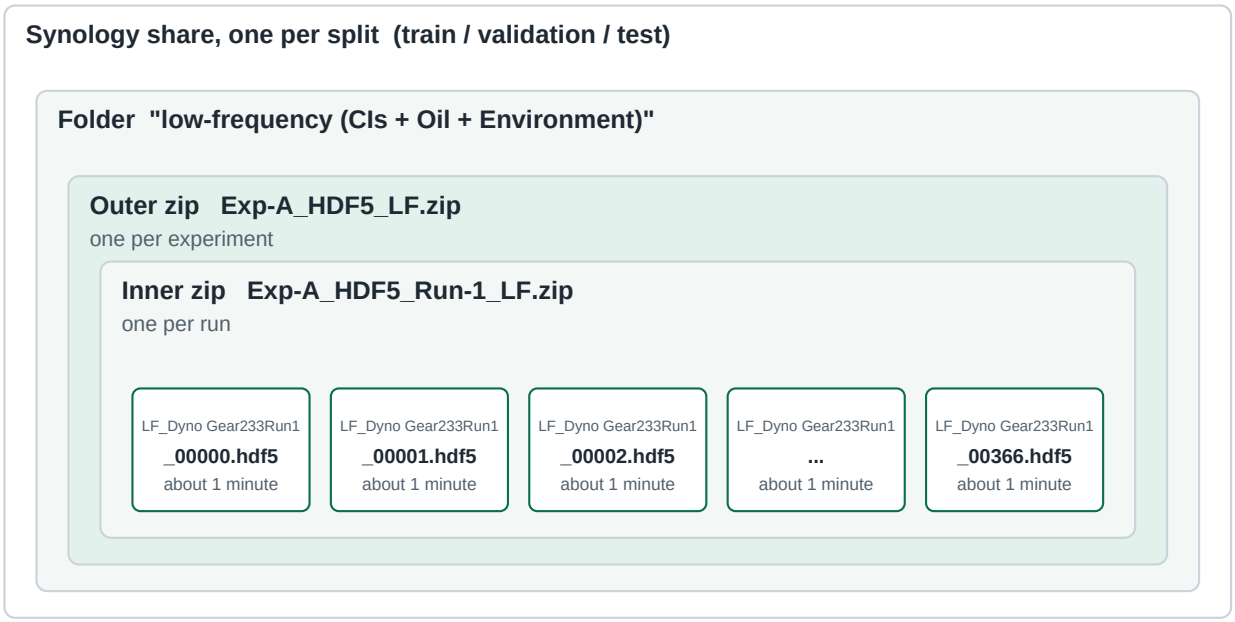
An **experiment** is one physical gear tested until it fails. Each experiment is broken into **runs**. A run is one continuous stretch of testing; between runs the rig was stopped, usually to photograph the gear teeth. Experiments have between 5 and 16 runs.

Inside each share there are up to four top-level folders:

Folder on the share	What is inside	Size
low-frequency (CIs + Oil + Environment)	One zip per experiment. Contains the per-minute HDF5 files with vibration metrics, oil debris counts, and machine state. **This is the folder the parquet tables are built from.**	hundreds of MB
low-frequency (CIs)	Same structure, but each HDF5 file holds only the vibration metrics. A lighter copy of the same information.	hundreds of MB
high-frequency	One zip per run. Holds the raw vibration waveforms sampled 100,000 times per second. Each zip is about 20 GB.	tens of GB per run
photos	One zip per run. Photographs of each gear tooth taken when the rig was stopped. Train split only.	about 1 GB
example_submission.csv	A template for the answer file. Lists a time in hours for each run end of each test experiment.	tiny

2. How the source files are nested

The low-frequency folders use three levels of packaging. An outer zip holds one inner zip per run, and each inner zip holds the HDF5 files for that run. The picture below shows the train share for experiment A, run 1.



The file names carry meaning, and the scripts read it:

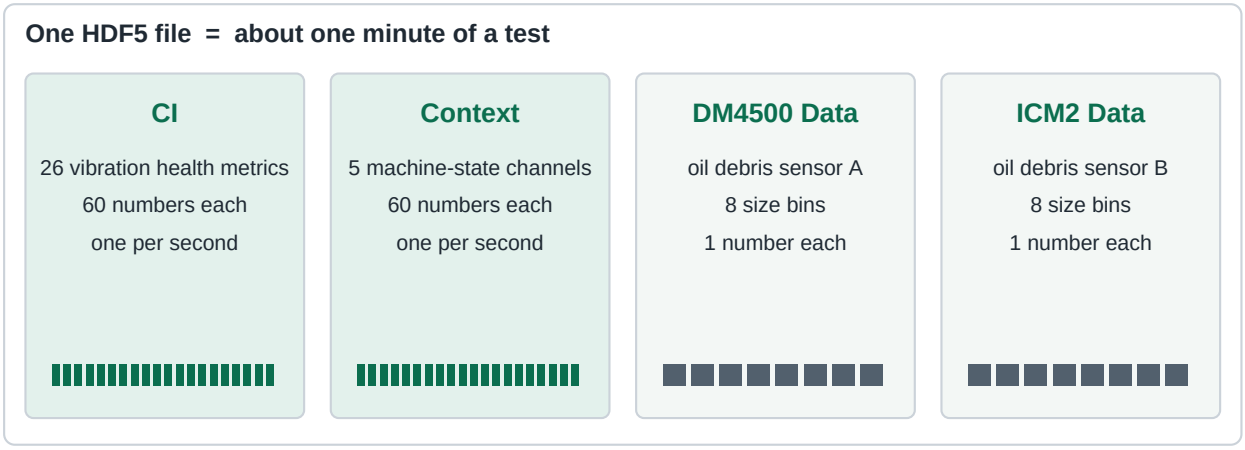
Part of the name	Example	Meaning
prefix	`LF_`	LF = full low-frequency file; CI = vibration-metrics-only file
session	`Dyno Gear233Run1`	The rig's own name for the session: gear serial 233, run 1
sequence	`_00042`	Position in time within the run: 0, 1, 2 ... one file per minute
extension	`.hdf5`	The file format (see section 3)

Most file names follow this pattern exactly. A few have an extra **_chunk_N** suffix, which means the rig split one minute into more than one file. The photo zips use a slightly different layout: inside each zip is a folder named after the run ("Run 1", "Break In", "0 Hours - Test Start"), then a folder per tooth, then the JPEG files.

One thing to watch: the run labels are not written the same way everywhere. The zip names say **Run-1**, the folders inside photo zips say **Run 1**, and some early photo sets are called **Break In** or **Pre Run**. The download script normalizes all of these to one spelling (Run-1, Break-In, Pre-Run, Test-Start) so files from the same run line up.

3. Inside one raw HDF5 file

HDF5 is a container format. Think of it as a small folder tree stored in a single file, where the leaves are arrays of numbers and every array can carry labels (called attributes). Each low-frequency file covers about one minute and has four groups.



Group	What it measures	Shape	Examples
CI	**Condition indicators.** 26 health metrics computed from the vibration signal, each over a one-second window. These are the main signals for detecting gear damage.	26 arrays of 60 numbers	rms, kurtosis, FM4, NA4, M6A, ALR, FLEET_WTMADE
Context	**Machine state.** How the rig was being driven at the time: shaft speed, torque, oil temperature, and overall vibration level on two accelerometers.	5 arrays of 60 numbers	PAU Speed (RPM), PAU Torque (ft-lbf), Temperature (degF), Accel1 RMS, Accel2 RMS (g)
DM4500 Data	**Oil debris, sensor A.** A count of metal particles seen in the oil, sorted into 8 size bins. One reading per file.	8 arrays of 1 number	DM4500 Bin 0 ... Bin 7
ICM2 Data	**Oil debris, sensor B.** Same idea, different instrument.	8 arrays of 1 number	ICM2 Bin 0 ... Bin 7

The high-frequency files have these same four groups plus the raw waveforms and some four-second metrics. The flattening script ignores those extra groups; only the four above feed the parquet tables.

Timing: every array knows when it starts

Each array carries three timing attributes: **wf_start_time** (an absolute timestamp), **wf_increment** (seconds between samples, which is 1), and **wf_start_offset** (a shift, in seconds, applied to the first sample). The time of sample number i is therefore:

```
time_of_sample(i) = wf_start_time + wf_start_offset + i * wf_increment
```

This matters because the four groups are **not** on the same clock:

- The **CI** arrays are stamped with an offset of 1 second. Each value describes the one-second window that *ended* at its stamp, not the one that began there.
- The **Context** arrays start about a quarter of a second after the CI arrays and have an offset of 0.
- The **oil debris** readings carry their own single timestamp, usually near the end of the minute.
- Arrays in the same file often have different lengths. 59 CI values next to 60 Context values is common.

Because of this, matching rows by position ("the 5th CI value goes with the 5th Context value") would be wrong. The flattening script matches on time instead. Section 5 shows how.

4. Step one: download and un-nest (phm_data.py)

This script talks to the Synology server, pulls the archives down, and turns the zip-in-zip structure into flat folders. It has two modes. The **small** mode handles everything except the high-frequency data. The **hf** mode handles the high-frequency zips separately because they are so large.

```
.venv/bin/python scripts/phm_data.py small
.venv/bin/python scripts/phm_data.py hf --split test --exp Exp-D --run Run-9
```

What the small mode does, in order

1. **Connect.** It opens the share for one split. For the test split it also sends the password and keeps the session cookie.
2. **Walk.** It asks the server for the full list of files, recursing into folders, skipping the high-frequency folder.
3. **Download.** Each remote file is saved under **data/raw/** with its original path, so the raw folder mirrors the share exactly. A file that already exists with the right size is skipped, so the script can be re-run safely.
4. **Copy the templates.** The example submission CSVs are copied straight into data/<split>/.
5. **Unzip twice.** For each low-frequency zip it opens the outer zip, then opens each inner zip in memory, then writes out every HDF5 file inside. Mac junk files (.DS_Store, __MACOSX, ._ prefixes, Thumbs.db) are dropped.
6. **Rename flat.** Each extracted file is saved as `data/<split>/<modality>/<Exp>_<Run>_<original name>`. Spaces in the original name become underscores. The experiment and run labels are prepended so the name alone tells you where the file belongs, and no two files can collide.
7. **Unpack photos.** Photo zips are handled the same way, except the run label is read from the folder inside the zip rather than the zip name, because one zip on the share is mislabeled.
8. **Write the manifest.** One CSV row per extracted file records its new path, modality, experiment, run, size, the zip it came from, and its full path inside that zip.

The word **modality** is just the data type: **lf** for the full low-frequency files, **ci** for the metrics-only files, **hf** for high-frequency, and **photos**.

Result on disk

```
data/
  raw/                        exact copies of the remote zips (provenance)
  channels.csv                units and settings for every column (from step two)
  train/
    manifest.csv              one row per extracted file, with where it came from
    lf/   Exp-A_Run-1_LF_Dyno_Gear233Run1_00000.hdf5 ...   11,584 files
    ci/   Exp-A_Run-1_CI_Dyno_Gear233Run1_00000.hdf5 ...   11,578 files
    photos/ Exp-A_Break-In_All_Teeth_Tooth_05.jpg ...        2,099 files
    lf_1s.parquet, particles.parquet                         (written by step two)
  validation/  same layout, experiment G, no photos
  test/        same layout, experiments D and E, plus an hf/ folder for one run
```

A sample manifest row looks like this. The last column is the trail back to the source:

```
file:        lf/Exp-A_Run-1_LF_Dyno_Gear233Run1_00000.hdf5
modality:    lf
experiment:  Exp-A
run:         Run-1
size:        108272
source_zip:  low-frequency (CIs + Oil + Environment)/Exp-A_HDF5_LF.zip
inner_path:  Exp-A_HDF5_Run-1_LF.zip/LF_Dyno Gear233Run1_00000.hdf5
```

The high-frequency mode

The high-frequency zips are around 20 GB each, and the server limits each download stream to a few megabytes per second. Rather than download a whole zip, the script opens it remotely: it reads only the byte ranges it needs, finds the table of contents at the end of the zip, and streams individual HDF5 files out with several parallel connections. The zips never touch local disk. It checks free space before each file and stops with a message if less than 5 GB would remain. Only one test run (Exp-D, Run-9) has been pulled so far.

5. Step two: from HDF5 to a flat table (flatten_hdf5.py)

This script reads every file in a split's **lf/** folder and produces the two parquet tables. It uses the manifest to know which files to read and what experiment and run each belongs to. Files are processed in parallel, then combined.

```
.venv/bin/python scripts/flatten_hdf5.py              # all three splits
.venv/bin/python scripts/flatten_hdf5.py --split test --workers 8
```

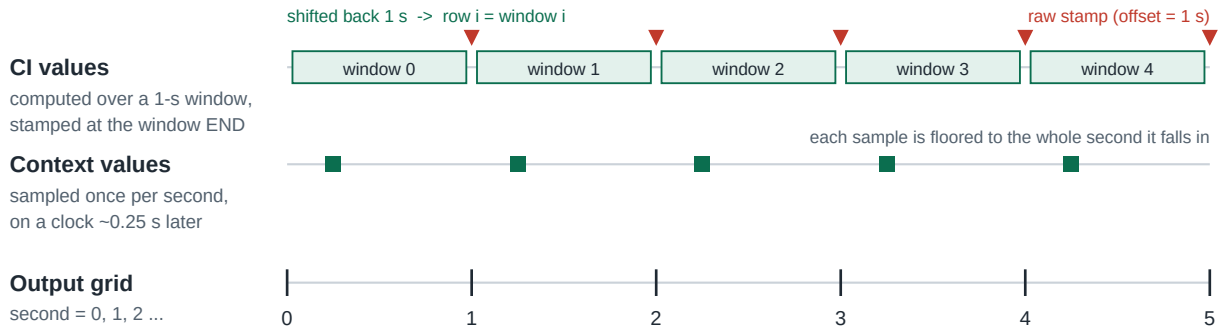
For each file

1. **Pull identifiers from the name.** Experiment and run come from the manifest. The session name (Gear233Run1), the sequence number, and the chunk number come from the original file name.
2. **Read every CI and Context array** and compute the absolute time of each value using the formula in section 3. For CI arrays, subtract one second first, so each value is labeled with the *start* of the window it describes rather than the end.
3. **Rename the columns.** HDF5 names like `CI/crest-TSA` and `Context/PAU Speed` become lowercase, underscore-separated names with a prefix: `ci_crest_tsa` and `ctx_pau_speed`.

4 **Snap everything to one-second slots.** Find the earliest timestamp in the file. For every value in every array, count how many whole seconds have passed since that anchor. That count becomes the row number. Two readings that fall in the same second land in the same row, even if their clocks differ by a fraction of a second.

5 **Join the columns side by side.** Each array becomes one column, indexed by that second number. Where an array has no value for a given second, the cell is left empty (null). This is how a 59-value CI array and a 60-value Context array coexist in one table without shifting.

6 **Read the oil debris counts.** The 16 bin values and their timestamp become a single row for the particles table.



Files that are truncated or empty (a few exist in the source) are skipped and reported at the end of the run rather than crashing the script.

Then, for the whole split

- **Stack** all the per-file tables into one and **sort** by experiment, run number, then timestamp. Sorting by run number (not the run label as text) keeps Run-10 after Run-9.
- **Add test_hours.** This is the running count of seconds that contain at least one vibration metric, within each experiment, divided by 3600. It is the rig's operating time in hours, and it lines up with the times used in the submission templates to within a few minutes. Rows with no vibration data (the rig was idle or the sensor was off) do not advance it.
- **Write** lf_1s.parquet and particles.parquet into the split folder.
- **Write channels.csv** once, at the top of data/. It lists every column, the HDF5 path it came from, its unit, and the processing settings stored on the source array (for example which accelerometer a metric was computed from, and the gear mesh frequency used).

6. The final tables

lf_1s.parquet: one row per second

Each row is one second of one file. The first ten columns say *when and where*; the remaining 31 are the measurements. Empty cells mean the sensor had no reading in that second.

Column	Type	Meaning
`experiment`	text	Gear being tested, e.g. Exp-A
`run`	text	Run label, e.g. Run-3
`run_num`	int	Run as a number, for sorting
`file`	text	Flat file name this row came from
`session`	text	The rig's session name, e.g. Gear233Run1
`file_seq`	int	Sequence number from the file name
`chunk`	int	Chunk number from the file name, 0 if none
`second`	int	Whole seconds since the earliest reading in this file
`timestamp`	datetime	Wall-clock time of this second
`test_hours`	float	Cumulative operating hours within the experiment
`ci_* (26 columns)`	float	Vibration condition indicators, one per second. See channels.csv for the list: ci_rms, ci_kurtosis, ci_fm0, ci_fm4, ci_fm4star, ci_m6a, ci_m8a, ci_na4, ci_alr, ci_e_ratio, ci_crest_tsa, ci_fleet_wtmade, ci_v_peak_mmps, and so on
`ctx_accel1_rms, ctx_accel2_rms`	float	Overall vibration level on each accelerometer, in g
`ctx_pau_speed`	float	Shaft speed, RPM
`ctx_pau_torque`	float	Applied torque, ft-lbf
`ctx_temperature`	float	Oil temperature, degrees F

particles.parquet: one row per file

Oil debris is read once a minute, so it gets its own table. Each row has the same seven identifier columns as above, a **timestamp** for the reading, and 16 integer counts: **dm4500_bin_0** through **dm4500_bin_7** and **icm2_bin_0** through **icm2_bin_7**. Bin 0 is the smallest particle size and bin 7 the largest. Join it to the per-second table on **file** when you need both.

How big the tables are

Split	HDF5 files read	lf_1s rows	lf_1s columns	particles rows
train	11,584	677,835	41	11,071
validation	5,889	356,514	41	5,864
test	6,618	396,746	41	6,593

Things worth knowing before you analyze

- Most files produce 59 or 60 rows. A handful produce far more because the rig paused mid-file and the clock kept running; the **second** column jumps accordingly. Filter on test_hours or timestamp gaps if this matters to you.

- About 0.3 percent of rows have empty vibration metrics and a similar share have empty context values. These are seconds where one clock had a reading and the other did not.
- **test_hours** resets to zero for each experiment, not each run. Run boundaries are visible through the **run** column.
- The CI columns are dimensionless health scores. Their meaning and settings live in channels.csv; the starred variants (fm4star, na4star, m6astar, m8astar) are normalized against a healthy baseline run.
- The parquet tables contain only what was in the **lf** folder. The ci folder is a subset of the same data, and the photos and high-frequency waveforms are not flattened.

Glossary

Term	Plain meaning
Dynamometer (dyno)	The test rig that spins the gear under a controlled load.
Experiment	One gear, tested from new until failure.
Run	One uninterrupted stretch of an experiment, between inspections.
Condition indicator (CI)	A single number that summarizes one second of vibration in a way that tends to rise as a gear degrades.
Context	What the rig was doing: speed, torque, temperature, overall vibration.
Oil debris bins	Counts of metal flakes in the oil, grouped by size. More and larger flakes mean more wear.
HDF5	A file format that stores many named arrays, with labels, inside one file.
Parquet	A compact column-oriented table format that pandas, Polars, and most tools read directly.
Split	One of train, validation, or test.
Modality	One of lf, ci, hf, or photos: the kind of data in a file.
Manifest	A CSV listing every extracted file and where on the share it came from.